Lecture 10 – Software Testing Strategies & White-Box Coverage

---

1. Role of Specification (FOUNDATION CONCEPT)

Key Idea

Testing is **meaningless without specifications**.

Why?

Because:

- You need to know **what is expected**
- Testing compares **actual behavior vs specified behavior**

Exam Line

> Testing cannot be meaningful unless we have a specification.

---

2. Levels of Specification vs Levels of Testing

Large software systems have **multiple levels of specifications**, and **each level has a corresponding test level**.

Levels of Specification

1. **Functional Specification**
    - What the system should do
    - User requirements
2. **Design Specification**
    - Architecture
    - Module interaction
3. **Detailed Design Specification**
    - How each module is implemented
    - Code-level details

Corresponding Levels of Testing

Specification Level	Testing Level
Functional spec	System testing
Design spec	Integration testing
Detailed design spec	Unit testing
User validation	Acceptance testing

📌 **Very common exam question:**

> Match specification level with testing level

---

3. Software Testing Strategies

(Big Bang vs Incremental Testing)

Big Bang Testing

Definition

All modules are combined and tested **at once**, after development is complete.

Advantages

- Simple
- No stubs or drivers

Disadvantages

- Bugs found very late
- Hard to locate error source
- High risk

📌 Exam line:

> Big bang testing delays defect detection.

Incremental Testing

Definition

Modules are tested **step by step**, as they are developed.

Process:

- Unit testing
- Integration testing
- System testing

Advantages

- Early defect detection
- Easier debugging
- Lower risk

Disadvantages

- Requires stubs and drivers
- More planning needed

📌 Exam line:

> Incremental testing is preferred despite higher effort.

---

4. Top-Down vs Bottom-Up Testing

(Stubs vs Drivers)

This part is **VERY EXAMINABLE**.

Top-Down Testing

How it works

- Start testing from **main module**
- Lower modules are tested later

Uses Stubs

- Stub = dummy module
- Replaces lower-level modules
- Returns predefined results

📌 Example:
Main module calls login module
Login module not ready → use stub

Bottom-Up Testing

How it works

- Start testing from **lowest modules**
- Main module tested last

Uses Drivers

- Driver = dummy calling program
- Calls lower-level modules

📌 Example:
Database module tested first
UI not ready → use driver

Stubs vs Drivers (Very Important Table)

Feature	Stub	Driver
Used in	Top-down testing	Bottom-up testing
Role	Called program	Calling program
Replaces	Lower module	Higher module

📌 **One-line exam rule**:

> Top-down uses stubs, bottom-up uses drivers.

---

5. Black-Box vs White-Box Testing

Black-Box Testing

Definition

Testing based on:

- inputs
- outputs
- requirements

Characteristics

- Internal code not visible
- Functional testing
- Requirement based

📌 Example:
Login works or not, without seeing code

White-Box Testing

Definition

Testing based on:

- internal logic
- code structure

Characteristics

- Code visible
- Done by developers
- Coverage oriented

📌 Example:
Testing conditions, loops, branches

---

6. White-Box Coverage Concepts (CORE OF LECTURE 10)

6.1 Path Coverage

Definition

Ensuring **all possible execution paths** are tested.

Important Point

- Practically impossible for large programs

📌 Example:
If a module has **10 conditions**,
possible paths = **2^{10} = 1024**

📌 Exam Line:

> Path coverage is impractical for large systems.

6.2 Statement (Line) Coverage

Definition

Ensures **every line of code executes at least once**.

Example

```
if (cond) {
 line1;
 line2;
 line3;
} else {
 line4;
}
```

If only `cond = true` is tested:

- Lines executed: 3 out of 4
- Statement coverage = 75%

📌 Limitation:

- Does not guarantee branch testing

6.3 Branch Coverage

Definition

Ensures **every branch (true/false)** is executed at least once.

Example

```
if (a > b)
 print("A");
else
 print("B");
```

Test cases:

- a > b
- a ≤ b

📌 Both branches executed → 100% branch coverage

6.4 Condition Coverage

Definition

Ensures **each condition inside a decision** evaluates to both **true and false**.

Example

```
if (a > 0 && b > 0)
 print("Hello");
```

```
else
 print("Bye");
```

Test cases for **condition coverage**:

- (1, 0)
- (0, 1)

📌 Condition coverage ≠ branch coverage

---

7. Branch Coverage vs Condition Coverage (EXAM FAVORITE)

Aspect	Branch Coverage	Condition Coverage
Focus	Entire decision	Individual conditions
Requires	True & false path	Each condition true & false
Stronger	❌	✅

📌 Exam trap:
Branch coverage does **not guarantee** condition coverage.

---

8. Coverage Summary Table (MEMORIZE)

Coverage Type	What it Covers
Statement	Lines of code
Branch	Decision outcomes
Condition	Boolean conditions

Path	All execution paths

---

1. Black Box Testing (Functionality Testing)

Definition (Exam-ready)

Black box testing is a testing technique that:

- ignores internal code structure
- focuses only on **inputs and outputs**
- is based on **software specifications**

📌 Tester does NOT know how the code works internally.

Key Characteristics

- Code is invisible
- Requirement-based
- Functional testing
- Used at system and acceptance levels

📌 Exam line:

> Black box testing identifies bugs through incorrect outputs, not through code inspection.

---

2. Equivalence Class Partitioning (ECP)

This is a **very important black-box technique**.

Definition

Equivalence Class Partitioning divides input data into **classes** such that:

- all values in a class behave the same
- testing one value represents the entire class

Types of Equivalence Classes

1. **Valid equivalence classes**
2. **Invalid equivalence classes**

📌 Rule:

> Each valid and each invalid equivalence class must be tested at least once.

Why ECP Is Used

- Reduces number of test cases
- Improves test efficiency
- Avoids redundant testing

Simple Example

Input range: `1–100`

Equivalence classes:

- Valid: 1–100
- Invalid: <1
- Invalid: >100

Instead of 100 tests → **3 tests only**

Exam Trap

- Equivalence classes are derived from **specification**, not code.

---

3. Boundary Value Analysis (BVA)

Definition

Boundary Value Analysis focuses on testing values at the **edges of equivalence classes**.

📌 Most bugs occur at boundaries.

Boundary Values Tested

For a range `1–100`:

- 0 (just below min)
- 1 (min)
- 2 (just above min)
- 99 (just below max)
- 100 (max)
- 101 (just above max)

Relationship with ECP

- BVA is an **extension of Equivalence Class Partitioning**
- Both are used together

📌 Exam line:

> Boundary testing comes after equivalence class partitioning.

Exam Trap

If question mentions:

- minimum
- maximum
- just above
- just below

→ **Boundary Value Analysis**

---

4. Advantages and Disadvantages of Black Box Testing

Advantages

- Requires fewer resources than white-box testing
- Allows system performance testing
- No need to understand code

Disadvantages

- No control over code coverage
- May miss internal logic errors
- Cannot verify coding standards

📌 Exam favorite:

> Black box testing cannot ensure line coverage.

---

5. Mutation Testing (Very Important & Often Ignored)

Purpose

Mutation testing evaluates the **quality of test cases**, not the code.

Definition

Mutation testing introduces **small changes (mutations)** into the source code to check whether test cases can detect the error.

Key Terms

- **Mutant**: Modified version of the program
- **Killed mutant**: Test case detects the mutation
- **Alive mutant**: Test case fails to detect mutation

Mutation Score (Exam Formula)

Mutation Score = (Killed Mutants / Total Mutants) × 100

Types of Mutations

- Operand replacement
- Operator replacement
- Statement deletion
- Logical connector change

Advantages

- Improves test quality
- Detects weak test cases
- Finds hidden faults

Disadvantages

- Very time-consuming
- Expensive
- Difficult to do manually

📌 Hence automation tools are used.

---

6. Relationship Between White Box and Black Box (Exam Concept)

Aspect	White Box	Black Box
Code visibility	Visible	Hidden
Focus	Internal logic	Functional behavior

Coverage	Yes	No
Used by	Developers	Testers
Based on	Code	Specification

📌 Exam line:

> White box tests structure, black box tests behavior.
